

TABLA DE CONTENIDO

NIVEL FUNDAMENTAL	1
Módulo 4. Objetos y Modelos de Datos	1
Clases y Objetos	1
Herencia, Composición y Métodos especiales	2
Dataclasses y Attrs	4
Pydantic para Validación y Serialización	6
Preparación del Laboratorio	7
Estructura del Laboratorio	8

NIVEL FUNDAMENTAL

MÓDULO 4. OBJETOS Y MODELOS DE DATOS

CLASES Y OBJETOS

Clase. Plantilla que define los datos llamados *atributos*, y los comportamientos definidos mediante *métodos*. Describe qué información y acciones tendrá un *producto*.

```
class Producto:
    def __init__(self, nombre, precio):
        self.nombre = nombre
        self.precio = precio

    def describir(self):
        return f"{self.nombre}: ${self.precio}"
```

- **Atributos de clase.** Un atributo de clase pertenece a la clase y puede *compartirse*. Los atributos específicos de cada objeto deben definirse normalmente dentro de *init*.

```
class Producto:
    categoria = "General"

    def __init__(self, nombre):
        self.nombre = nombre
```

Objeto. Instancia concreta de una clase. Se puede acceder a sus atributos y llamar sus métodos.

```
producto = Producto("Teclado", 50)
```

donde

Estructura	Acceso a atributos	Llamada a métodos
Producto: clase. producto: objeto. "Teclado": valor del atributo nombre. 50: valor del atributo precio.	<code>print(producto.nombre)</code> <code>print(producto.precio)</code>	<code>print(producto.describir())</code>

- **Atributos de instancia.** Cada objeto puede tener sus propios valores (atributos).

Métodos. Es una función definida dentro de una clase. Para llamarlo, el objeto se utiliza como primer argumento.

```
def describir(self):
    return f"{self.nombre}: ${self.precio}"
[...]  
producto.describir()
```

Método ‘__init__’. Se ejecuta automáticamente al crear un objeto. Recibe los valores iniciales y los asigna al objeto mediante *self* (instancia actual).

```
def __init__(self, nombre, precio):
    self.nombre = nombre
    self.precio = precio
```

En Java, ‘__init__’ es parecido al constructor.

Resumiendo:

Atributo de clase	Pertenece a la clase
Atributo de instancia	Pertenece a cada objeto
Parámetro	Valor recibido por un método
Método	Comportamiento de la clase

HERENCIA, COMPOSICIÓN Y MÉTODOS ESPECIALES

Herencia. Permite crear una clase **a partir de otra**; se hereda el atributo y el método, y permite agregar más de ellos.

```
class Producto:
    def __init__(self, nombre):
        self.nombre = nombre

    def describir(self):
        return self.nombre

class ProductoDigital(Producto):
    def descargar(self):
        return f"Descargando {self.nombre}"
```

Sobreescritura y super(). Una clase hija puede **redefinir** un método, así como **extender** el constructor de la clase padre. `super()` permite utilizar la implementación de la clase padre.

Sobreescritura	Extensión
<pre>class ProductoDigital(Producto): def describir(self): return f"Producto digital: {self.nombre}"</pre>	<pre>class ProductoDigital(Producto): def __init__(self, nombre, tamaño_mb): super().__init__(nombre) self.tamaño_mb = tamaño_mb</pre>

Composición. Es construir una clase utilizando objetos de otras clases.

```
class Direccion:
    def __init__(self, ciudad):
        self.ciudad = ciudad
class Cliente:
    def __init__(self, nombre, direccion):
        self.nombre = nombre
        self.direccion = direccion
```

Ejemplo

Dos clases diferentes.

```
direccion = Direccion("Madrid")
cliente = Cliente("Ana", direccion)

print(cliente.direccion.ciudad)
```

Uso

Cliente no hereda de dirección, sino que contiene una instancia

ProductoDigital es un **Producto**.
Cliente tiene una **Direccion**.

Explicación

- **Herencia.** Relación “**es un**”.
- **Composición.** Relación “**tiene un**”.

Métodos especiales o ‘dunder methods’. Métodos cuyos nombres empiezan y terminan con **dos guiones bajos**.

__str__
Define la representación legible para el usuario.

```
class Producto:
    def __init__(self, nombre):
        self.nombre = nombre

    def __str__(self):
        return self.nombre
```

__repr__
Define una representación útil para depuración.

```
class Producto:
    def __repr__(self):
        return f"Producto(nombre={self.nombre!r})"
```

__eq__ Define cuándo dos objetos se consideran iguales.	<pre>class Producto: def __init__(self, codigo): self.codigo = codigo def __eq__(self, otro): if not isinstance(otro, Producto): return NotImplemented return self.codigo == otro.codigo</pre>
---	--

__lt__ Permite definir el operador '<'.	<pre>class Producto: def __init__(self, precio): self.precio = precio def __lt__(self, otro): return self.precio < otro.precio</pre>
---	--

Resumiendo:

Concepto	Relación	Forma Básica	Uso Principal
Herencia	<i>“Es un”</i>	Class Hija(Padre):	Reutilizar y especializar una clase existente.
Sobreescritura	La clase hija <i>reemplaza</i> un comportamiento heredado	Definir en la hija un método con el mismo nombre.	Cambiar o ampliar el comportamiento del padre.
Composición	<i>“Tiene un”</i>	Recibir y guardar un objeto como atributo.	Construir objetos combinando otras clases.

DATACLASSES Y ATTRS

Una **dataclass** genera automáticamente parte del código de una clase que representa principalmente datos, para evitar lo repetitivo de escribir manualmente `__init__`, `__repr__` y comparaciones.

Uso	Explicación
<pre>from dataclasses import dataclass @dataclass class Producto: nombre: str precio: float producto = Producto("Teclado", 50.0) print(producto)</pre>	dataclass Básica: La clase genera automáticamente: ▪ <code>__init__</code> ▪ <code>__repr__</code> ▪ <code>__eq__</code> Salida: <pre>Producto(nombre='Teclado', precio=50.0)</pre>

```
@dataclass
class Producto:
    nombre: str
    precio: float
    producto = Producto("Teclado",
                        "cincuenta")
```

Anotaciones:

Documentan los tipos esperados, pero no validan los valores en tiempo de ejecución.

```
@dataclass
class Producto:
    nombre: str
    precio: float
    disponible: bool = True
```

Valores Predeterminados:

*Los campos obligatorios deben declararse **antes** de los campos con valores predeterminados.*

Campos calculados. Para **excluir un atributo** del constructor y calcularlo después.

```
from dataclasses import dataclass, field
@dataclass
class Producto:
    precio: float
    cantidad: int
    total: float = field(init=False)

    def __post_init__(self):
        self.total = self.precio * self.cantidad

producto = Producto(10.0, 3)
print(producto.total)
```

Donde `__post_init__` se ejecuta después del `__init__` generado.

Inmutabilidad. Crear una `dataclass` inmutable evita la **modificación** de sus **atributos**, útil para objetos que representan valores fijos.

```
@dataclass(frozen=True)
class Coordinada:
    x: float
    y: float
```

Ordenamiento. Habilita las **comparaciones de orden**, basado en los campos de la clase. Así, Python puede generar métodos como: `__lt__`, `__le__`, `__gt__` y `__ge__`.

```
@dataclass(order=True)
class Producto:
    precio: float
```

'attrs'. Biblioteca externa para definir clases de datos de forma declarativa. Al igual que `dataclass`, genera automáticamente métodos comunes.

```
from attrs import define

@define
class Producto:
    nombre: str
    precio: float
```

Puede ofrecer herramientas adicionales para:

- Validadores.
- Convertidores.
- Configuración avanzada de atributos.
- Clases mutables o inmutables.

Diferencia entre `dataclasses` y `attrs`

Característica	<code>dataclasses</code>	<code>attrs</code>
<i>Pertenece a biblioteca estándar</i>	Sí	No
<i>Requiere instalación externa</i>	No	Sí
<i>Genera métodos comunes</i>	Sí	Sí
<i>Validación integrada avanzada</i>	Limitada	Más flexible
<i>Adecuada para modelos sencillos</i>	Sí	Sí
<i>Opciones avanzadas</i>	Menos	Más

PYDANTIC PARA VALIDACIÓN Y SERIALIZACIÓN

Pydantic. *Biblioteca* para definir modelos que validan datos de entrada, convierten tipos compatibles, serializan objetos y generan errores detallados.

```
from pydantic import BaseModel
```

```
class ProductIn(BaseModel):
    name: str
    price: float
```

- A diferencia de las anotaciones de una `dataclass`, Pydantic sí realiza validación en tiempo de ejecución. “*Valida al crear el modelo*”.
- Su dependencia se agrega a ***pyproject.toml*** (laboratorio).

Validación. Produce un ***ValidationError*** para restringir el uso de los atributos.

```
from pydantic import BaseModel, Field
```

```
class ProductIn(BaseModel):
    name: str = Field(min_length=1)
    price: float = Field(gt=0)
```

Donde ahora:

- `name` no puede estar vacío.
- `price` debe ser mayor que cero.

Conversión de tipos. Pydantic puede convertir algunos valores compatibles.

Serialización. Puede convertir un modelo a diccionario o un JSON.

A diccionario

A JSON

```
datos = producto.model_dump() texto_json = producto.model_dump_json()
```

Modelos de entrada y salida. Separar los modelos según su *propósito*, por ejemplo, datos recibidos (`OrderIn`) y datos por devolver (`OrderOut`), incluyendo cálculos realizados.

Validadores. Es posible definir validaciones personalizadas (`@field_validator`). Los validadores se ejecutan al crear el modelo.

```
from pydantic import BaseModel, field_validator

class OrderIn(BaseModel):
    quantity: int

    @field_validator("quantity")
    @classmethod
    def quantity_must_be_positive(cls, value):
        if value <= 0:
            raise ValueError("La cantidad debe ser mayor que cero")

        return value
```

Diferencias entre Pydantic y dataclass

Característica	dataclasses	attrs
<i>Biblioteca estándar</i>	Sí	No
<i>Crea clases de datos</i>	Sí	Sí
<i>Valida en ejecución</i>	Limitado	Sí
<i>Serializa fácilmente</i>	Requiere trabajo adicional	Sí
<i>Ideal para entidad interna</i>	Sí	Puede serlo
<i>Ideal para entradas externas</i>	No siempre	Sí

PREPARACIÓN DEL LABORATORIO

El objetivo es separar:

- *Entrada externa:* modelo Pydantic `OrderIn`.
 - *Entidad interna:* `dataclass` `Order`.
 - *Salida serializable:* modelo Pydantic `OrderOut`.
- **Diseño de Order.** Entidad que representará una orden con: identificador, nombre del producto, precio unitario, cantidad y porcentaje de impuesto. Tendrá por cálculos derivados:

```

subtotal = precio_unitario × cantidad
impuesto = subtotal × porcentaje_impuesto
total = subtotal + impuesto

```

- **Diseño de `OrderIn`.** Validará los datos recibidos de `Order`.
- **Diseño de `OrderOut`.** Representará la información que devuelve. Será serializable mediante `model_dump()` (diccionario) o `model_dump_json()` (JSON).
- **Conversión de entrada a entidad.** Mediante una función de conversión, para transformar el modelo validado `OrderIn` en una instancia de `Order`. El flujo será:

```

Datos externos
  ↓
OrderIn
  ↓ validación
Order
  ↓ cálculos derivados
OrderOut
  ↓ serialización
Diccionario o JSON

```

- **Comparaciones.** La dependencia `Order` podrá comparar con otra orden por su total. Se implementará mediante un método especial como `__lt__`, y también tendrá igualdad con `__eq__`, ya sea por el identificador o los datos completos.

ESTRUCTURA DEL LABORATORIO

Flujo de datos:

```

OrderIn (Datos validados por Pydantic)
  |
  | Datos validados
  ▼
Order (Entidad interna que calcula subtotal, tax y total)
  |
  | Cálculos derivados
  ▼
OrderOut (Salida serializable para devolver como diccionario o JSON)
  |
  | Serialización
  ▼
JSON

```